DO NOW — RECALL

Advanced Big O and space complexity

Name: _____ Class: _____ Date: _____

1. What does O(log n) indicate?

2. What is auxiliary space?

3. Is Big O an exact millisecond prediction?

4. A sorted array of length64 is reduced by half after each unsuccessful search step. How many halvings reach length1?

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

Work growing logarithmically, as with repeated halving.

2. Model answer

Extra memory beyond input storage.

3. Model answer

No. It classifies asymptotic growth.

4. Model answer

Six: 64→32→16→8→4→2→1.

By the end, you can…

• Identify the dominant operation

• Separate time and space

• Interpret asymptotics

Vocabulary — know these terms

asymptotic — Behaviour as input size grows.

Big O — Asymptotic upper bound.

auxiliary space — Storage additional to input.

polynomial — Growth such as n^2.

exponential — Growth such as 2^n.

WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

Guided 1 — original CHECK

Explain the method

New array b stores n values, giving O(n) auxiliary memory.

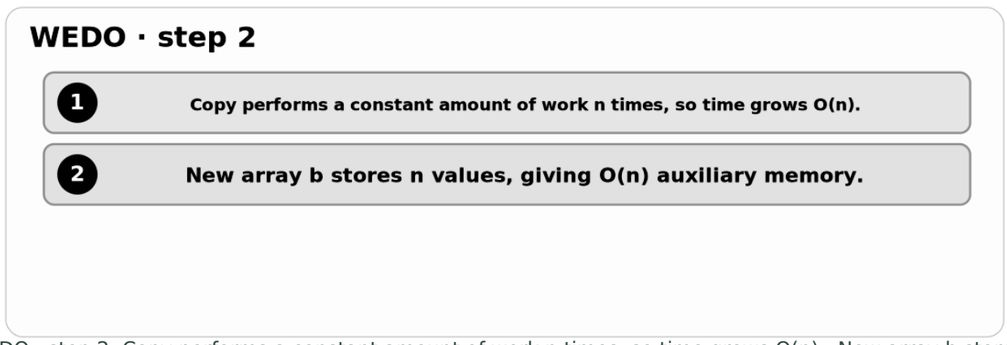
IMPROVE: Explain one correction to your own reasoning.

We Do Check · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Find time and extra space for copying each element to a separate new n-element array.

Copy performs a constant amount of work n times, so time grows O(n).



WEDO · step 2; Copy performs a constant amount of work n times, so time grows O(n).; New array b stores n values, giving O(n) auxiliary memory.

NEW LEARNING

Read, annotate and explain

Identify the dominant operation

Single full loop grows O(n); two full nested n loops grow O(n²). Repeated halving grows O(log n).

Separate time and space

Counting two scalar variables uses O(1) auxiliary memory even if repeated n² times; copying n elements uses O(n) extra storage.

Interpret asymptotics

Big O describes a bound on growth, not an exact runtime. Constants, cache effects and data size still matter.

Explain one step in your own words, using the correct technical terms.

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

New Learning

Advanced Big O and space complexity · stage 1

1. **n=16 → O(log₂n)≈4; O(n)=16**

Advanced Big O and space complexity · stage 1; n=16 → O(log₂n)≈4; O(n)=16

New Learning

Advanced Big O and space complexity · stage 2

1. **n=16 → O(log₂n)≈4; O(n)=16**
2. **n=16 → O(n log₂n)=64**

Advanced Big O and space complexity · stage 2; n=16 → O(log₂n)≈4; O(n)=16; n=16 → O(n log₂n)=64

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

An O(n²) algorithm must always use O(n²) memory.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

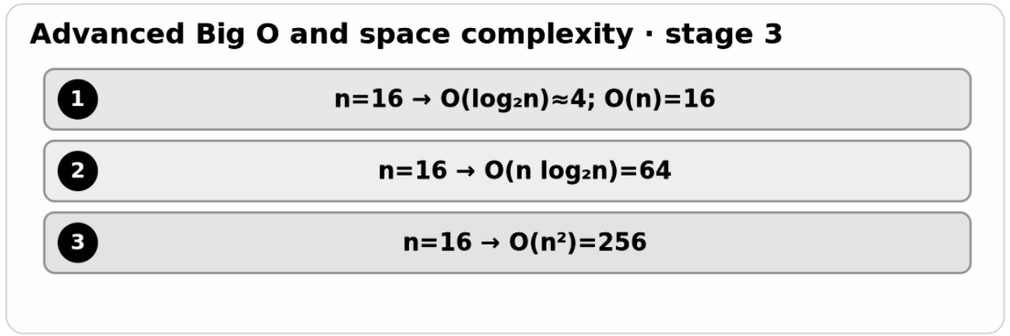
Question 3

Compare bubble sort and merge sort at large n using time and space trade-offs. [6]

New Learning · original lesson diagrams

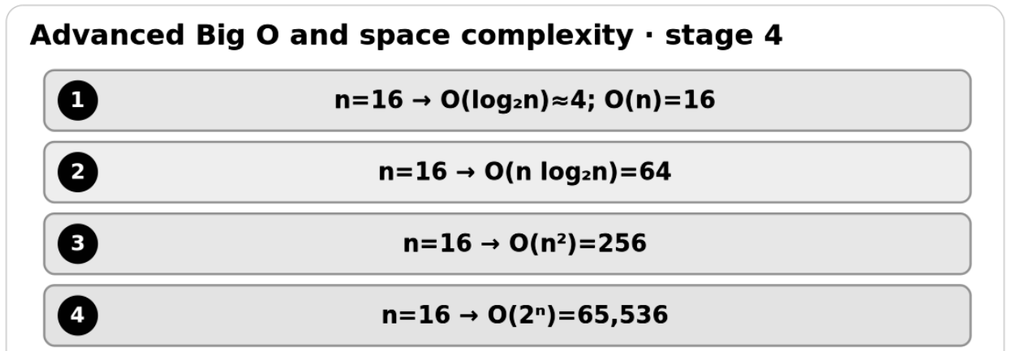
Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



Advanced Big O and space complexity · stage 3; n=16 → O(log₂n)≈4; O(n)=16; n=16 → O(n log₂n)=64; n=16 → O(n²)=256

New Learning



Advanced Big O and space complexity · stage 4; n=16 → O(log₂n)≈4; O(n)=16; n=16 → O(n log₂n)=64; n=16 → O(n²)=256; n=16 → O(2ⁿ)=65,536

I DO — WORKED EXAMPLE

Follow the worked method

Find worst-case time and auxiliary space for summing n array elements into one accumulator.

Predict first

Each element is visited once: time O(n).

Explain the method

The only additional state is a scalar sum (apart from iterator control): auxiliary space O(1).

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

Classify loops running once, n times, n^2 times and repeatedly halving n. [4]

Question 2

A function makes a new copy of every one of n elements. Identify time and auxiliary space. [6]

WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

Guided 1: Predict first Copy performs a constant amount of work n times, so time grows O(n).

DISCUSS — Compare your methods. Justify one important decision, identify one difference, and agree a specific improvement.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Find worst-case time and auxiliary space for summing n array elements into one accumulator.

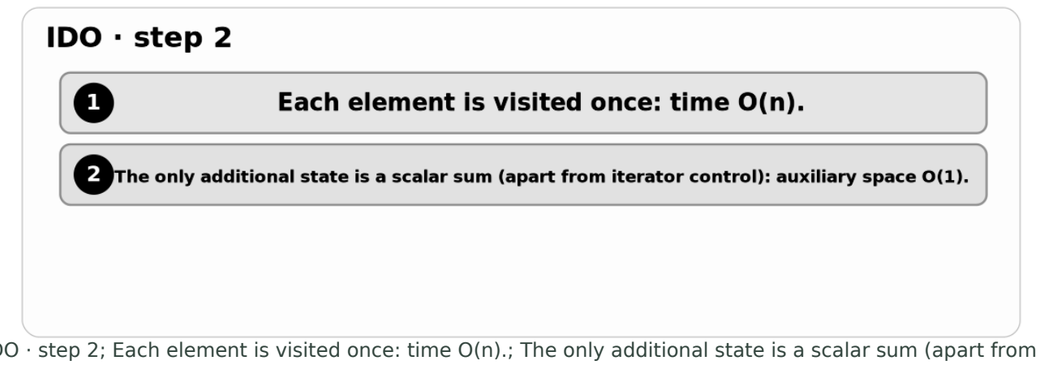
Each element is visited once: time O(n).



IDO · step 1; Each element is visited once: time O(n).

Find worst-case time and auxiliary space for summing n array elements into one accumulator.

Each element is visited once: time O(n).



IDO · step 2; Each element is visited once: time O(n).; The only additional state is a scalar sum (apart from iterator control): auxiliary space O(1).

WE DO — GUIDED PRACTICE

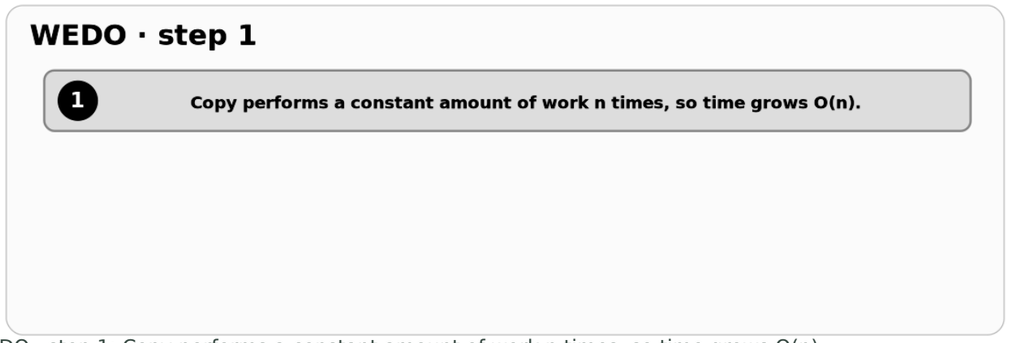
Try → Discuss → Improve

We Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Find time and extra space for copying each element to a separate new n-element array.

Copy performs a constant amount of work n times, so time grows O(n).



WEDO · step 1; Copy performs a constant amount of work n times, so time grows O(n).